

Section C Mini Capstone Project

Mini Project: Food Delivery Platform — Backend Database

Objective: Build the complete backend database for a Food Delivery application, combining DDL, DML, JOINS, Views, and PL/SQL into a single integrated .sql script.

- Create three tables: restaurants (restaurant_id, name, city, cuisine_type, created_at), menu_items (item_id, restaurant_id, item_name, price, category) with a FOREIGN KEY on restaurant_id, and orders (order_id, customer_name, restaurant_id, item_id, quantity, total_amount, order_date) with a FOREIGN KEY on restaurant_id — all with appropriate PRIMARY KEY and NOT NULL constraints.

```
create table restaurants (  
    restaurant_id integer primary key auto_increment,  
    name varchar(100) not null,  
    city varchar(100) not null,  
    cuisine_type varchar(100) not null,  
    created_at datetime not null default current_timestamp);  
  
create table menu_items (  
    item_id integer primary key auto_increment,  
    restaurant_id integer not null,  
    item_name varchar(100) not null,  
    price real not null,  
    category varchar(100) not null,  
    foreign key (restaurant_id) references restaurants(restaurant_id));  
  
create table orders (  
    order_id integer primary key auto_increment,
```

**customer_name varchar(100) not null,
restaurant_id integer not null,
item_id integer not null,
quantity integer not null,
total_amount real not null,
order_date datetime not null default current_timestamp,
foreign key(restaurant_id) references restaurants(restaurant_id));**

- **Populate with at least 3 restaurants, 4 menu categories, and 15 order records distributed across restaurants.**

insert into restaurants (name, city, cuisine_type) values

('Honest', 'Ahmedabad', 'Indian'),

('Dominos', 'Rajkot', 'Italian'),

('Sankalp', 'Surat', 'South Indian');

insert into menu_items

(restaurant_id, item_name, price, category) values

(1, 'Paneer Butter Masala', 250.00, 'Main Course'),

(1, 'Veg Biryani', 180.00, 'Main Course'),

(1, 'Masala Dosa', 120.00, 'Main Course'),

(1, 'Cold Coffee', 100.00, 'Beverage'),

(1, 'Gulab Jamun', 80.00, 'Dessert'),

(2, 'Margherita Pizza', 299.00, 'Pizza'),

(2, 'Farmhouse Pizza', 399.00, 'Pizza'),

(2, 'Garlic Bread', 149.00, 'Starter'),

(2, 'Coke', 60.00, 'Beverage'),

(2, 'Choco Lava Cake', 99.00, 'Dessert'),

(3, 'Masala Dosa', 140.00, 'Main Course'),

(3, 'Idli Sambar', 100.00, 'Main Course'),

(3, 'Uttapam', 160.00, 'Main Course'),

(3, 'Lassi', 90.00, 'Beverage'),

(3, 'Ice Cream', 70.00, 'Dessert');

**insert into orders(customer_name, restaurant_id, item_id, quantity,
total_amount, order_date)values**

('Mital', 1, 1, 2, 500.00, '2026-10-01 12:30:00'),

('Nidhi', 1, 2, 1, 180.00, '2026-10-01 13:00:00'),

('Teja', 1, 4, 2, 200.00, '2026-10-02 10:00:00'),

('Mital', 1, 5, 3, 240.00, '2026-10-02 19:00:00'),

('Nidhi', 1, 3, 2, 240.00, '2026-10-03 12:00:00'),

('Teja', 2, 6, 1, 299.00, '2026-10-03 13:30:00'),

('Mital', 2, 7, 2, 798.00, '2026-10-04 19:30:00'),

('Nidhi', 2, 8, 1, 149.00, '2026-10-04 20:00:00'),

('Teja', 2, 9, 3, 180.00, '2026-10-05 11:00:00'),

('Mital', 2, 10, 2, 198.00, '2026-10-05 20:30:00'),

('Nidhi', 3, 11, 2, 280.00, '2026-10-06 12:00:00'),

('Teja', 3, 12, 3, 300.00, '2026-10-06 13:00:00'),

('Mital', 3, 13, 1, 160.00, '2026-10-07 19:00:00'),

('Nidhi', 3, 14, 2, 180.00, '2026-10-08 15:00:00'),

('Teja', 3, 15, 2, 140.00, '2026-10-08 20:00:00');

restaurant_id	name	city	cuisine_type	created_at
1	Honest	Ahmedabad	Indian	2026-10-10 23:06:04
2	Dominos	Rajkot	Italian	2026-10-10 23:06:04
3	Sankalp	Surat	South Indian	2026-10-10 23:06:04
NULL	NULL	NULL	NULL	NULL

item_id	restaurant id	item name	price	category
1	1	Masala	250	Main Course
2	1	Veg Biryani	180	Main Course
3	1	Masala Dosa	120	Main Course
4	1	Cold Coffee	100	Beverage
5	1	Gulab Jamun	80	Dessert
6	2	Margherita Pizza	299	Pizza
7	2	Farmhouse Pizza	399	Pizza
8	2	Garlic Bread	149	Starter
9	2	Coke	60	Beverage
10	2	Choco Lava Cake	99	Dessert
11	3	Masala Dosa	140	Main Course
12	3	Idli Sambar	100	Main Course
13	3	Uttapam	160	Main Course
14	3	Lassi	90	Beverage
15	3	Ice Cream	70	Dessert

category

Main Course

Beverage

Dessert

Pizza

Starter

order_id	customer_name	restaurant_id	item_id	quantity	total_amount	order_date
1	Mital	1	1	2	500	2026-10-01 12:30:00
2	Nidhi	1	2	1	180	2026-10-01 13:00:00
3	Teja	1	4	2	200	2026-10-02 10:00:00
4	Mital	1	5	3	240	2026-10-02 19:00:00
5	Nidhi	1	3	2	240	2026-10-03 12:00:00
6	Teja	2	6	1	299	2026-10-03 13:30:00
7	Mital	2	7	2	798	2026-10-04 19:30:00
8	Nidhi	2	8	1	149	2026-10-04 20:00:00
9	Teja	2	9	3	180	2026-10-05 11:00:00
10	Mital	2	10	2	198	2026-10-05 20:30:00
11	Nidhi	3	11	2	280	2026-10-06 12:00:00
12	Teja	3	12	3	300	2026-10-06 13:00:00
13	Mital	3	13	1	160	2026-10-07 19:00:00
14	Nidhi	3	14	2	180	2026-10-08 15:00:00
15	Teja	3	15	2	140	2026-10-08 20:00:00

restaurant_id	total_orders
1	5
2	5
3	5

restaurant_name	total_orders	total_revenue
Honest	5	1360
Dominos	5	1624
Sankalp	5	1060

- Create a VIEW named `restaurant_sales_summary` showing restaurant name, total number of orders, and total revenue grouped by restaurant.
- Write a stored procedure named `add_order` that accepts `customer_name`, `restaurant_id`, `item_id`, and `quantity`, calculates `total_amount` from the menu item price, validates the restaurant exists (ROLLBACK if not), and inserts the order record.

delimiter //

```
create procedure add_order(
    in p_customer_name varchar(100),
```

```
in p_restaurant_id integer,  
in p_item_id integer,  
in p_quantity integer)  
begin  
    declare v_price real;  
    declare v_total_amount real;  
    declare v_restaurant_count integer default 0;  
  
    start transaction;  
  
    select COUNT(*)  
    into v_restaurant_count  
    from restaurants  
    where restaurant_id = p_restaurant_id;  
  
    if v_restaurant_count = 0 then  
        rollback;  
        signal sqlstate '45000'  
        set message_text = 'Restaurant does not exist';  
    else  
        select price  
        into v_price  
        from menu_items
```

```
where item_id = p_item_id
    and restaurant_id = p_restaurant_id;
set v_total_amount = v_price * p_quantity;
insert into orders (
    customer_name,
    restaurant_id,
    item_id,
    quantity,
    total_amount
)
values (
    p_customer_name,
    p_restaurant_id,
    p_item_id,
    p_quantity,
    v_total_amount
);

commit;

end if ;

end //
```



```
delimiter ;
```

call add_order('Mrugakshi', 1, 2, 3);

select * from orders;

order_id	customer_name	restaurant_id	item_id	quantity	total_amount	order_date
1	Mital	1	1	2	500	2026-10-01 12:30:00
2	Nidhi	1	2	1	180	2026-10-01 13:00:00
3	Teja	1	4	2	200	2026-10-02 10:00:00
4	Mital	1	5	3	240	2026-10-02 19:00:00
5	Nidhi	1	3	2	240	2026-10-03 12:00:00
6	Teja	2	6	1	299	2026-10-03 13:30:00
7	Mital	2	7	2	798	2026-10-04 19:30:00
8	Nidhi	2	8	1	149	2026-10-04 20:00:00
9	Teja	2	9	3	180	2026-10-05 11:00:00
10	Mital	2	10	2	198	2026-10-05 20:30:00
11	Nidhi	3	11	2	280	2026-10-06 12:00:00
12	Teja	3	12	3	300	2026-10-06 13:00:00
13	Mital	3	13	1	160	2026-10-07 19:00:00
14	Nidhi	3	14	2	180	2026-10-08 15:00:00
15	Teja	3	15	2	140	2026-10-08 20:00:00
16	Mrugakshi	1	2	3	540	2026-10-10 23:51:42

- Write a TRIGGER named after_order_insert that logs a record into an order_audit table (order_id, restaurant_id, action='INSERT', log_time=NOW()) immediately after every new order is created.

delimiter //

create trigger after_order_insert

after insert on orders

for each row

begin

```
insert into order_audit (  
    order_id,  
    restaurant_id,  
    action,  
    log_time)  
values (  
    new.order_id,  
    new.restaurant_id,  
    'INSERT',  
    now()  
);  
end //  
  
delimiter ;  
  
call add_order('Mrugakshi', 1, 2, 3);  
  
select * from order_audit;
```

audit_id	order_id	restaurant_id	action	log_time
1	17	1	INSERT	2026-10-11 00:02:30
NULL	NULL	NULL	NULL	NULL